

1. 1951 - 1955

- Programación simbólica (uso de mnemónicos en lugar de código máquina puro): Assembly

1956 - 1960

- Primeros lenguajes de alto nivel y programación científica: FORTRAN
- Programación funcional: LISP
- Estructura y sintaxis formal: ALGOL 60

1961 - 1965

- Procesamiento de datos comerciales: COBOL
- Manejo de strings avanzado: SNOBOL
- Lenguajes orientados a aplicaciones específicas: JOVIAL
- Consolidación de estructuras de control: ALGOL 60

1966 - 1970

- Inicio de la programación orientada a objetos: SIMULA 67
- Lenguajes educativos y accesibles: BASIC
- Lenguajes multipropósito: PL/I
- Notación matemática compacta: APL

1971 - 1975

- Programación estructurada: Pascal
- Programación de sistemas y eficiencia: C
- Programación funcional moderna: Scheme
- Programación lógica: Prolog

1976 - 1980

- Orientación a objetos completa: Smalltalk
- Lenguajes seguros y tipados para sistemas críticos: ADA
- Sistemas de tipo avanzados: ML

1981 - 1985

- Consolidación de OOP: Smalltalk 80
- Desarrollo rápido (IDE, compiladores eficientes): Turbo Pascal
- Lenguajes para gráficos e impresión: PostScript

1986 - 1990

- Programación orientada a objetos en lenguajes populares: C++
- Programación genérica: C++
- Mejoras en programación científica (arrays, modularidad): FORTRAN 90

- Tipado fuerte y formal: SML

1991 - 1995

- Lenguaje de scripting: Perl, TCL
- Automatización de tareas: Perl
- Nacimiento de la Web: HTML

1996 - 2000

- Prog. orientada a objetos portable y ejecución en máquina virtual: Java
- Programación para la web (cliente): JavaScript
- Lenguaje de marcado estructurado: XML

2. Historia del lenguaje Java

El lenguaje de programación Java fue desarrollado a principios de la década de 1990 por James Gosling junto a un equipo de la empresa Sun Microsystems. El proyecto, conocido inicialmente como *Oak*, tenía como objetivo crear un lenguaje portátil para dispositivos electrónicos.

En 1995, Java fue presentado oficialmente y rápidamente ganó popularidad gracias a su lema **“Write Once, Run Anywhere”** (escribir una vez, ejecutar en cualquier lugar). Esto fue posible gracias a la **Máquina Virtual de Java (JVM)**, que permite ejecutar programas en distintos sistemas operativos sin necesidad de modificarlos.

Durante finales de los años 90 y principios de los 2000, Java se convirtió en uno de los lenguajes más utilizados para el desarrollo de aplicaciones empresariales, web y móviles. También fue clave en el desarrollo de aplicaciones para Internet.

En 2010, Oracle Corporation adquirió Sun Microsystems y pasó a encargarse del desarrollo y mantenimiento de Java. Desde entonces, el lenguaje ha seguido evolucionando, incorporando mejoras en rendimiento, seguridad y nuevas características como programación funcional (expresiones lambda).

Hoy en día, Java sigue siendo uno de los lenguajes más importantes y utilizados en el mundo del desarrollo de software.

3. Un buen lenguaje de programación debería tener una combinación de atributos que faciliten tanto el desarrollo como el mantenimiento del software. Los más importantes:

- **Simplicidad y legibilidad:** debe ser fácil de aprender y el código fácil de entender. Evita estructuras complicadas o ambiguas.

- **Claridad en los binding:** cómo se asocian variables con tipos y valores. Debe ser claro cuándo y cómo se realizan esas asociaciones.
- **Confiabilidad:** debe ayudar a evitar errores y comportamientos inesperados. Tipado fuerte, manejo de errores, control de memoria.
- **Soporte:** Disponibilidad de herramientas, documentación y comunidad. Compiladores, IDEs, librerías, comunidad activa.
- **Abstracción:** permite manejar la complejidad ocultando detalles innecesarios. Uso de funciones, clases, módulos.
- **Ortogonalidad:** las características del lenguaje se combinan de forma consistente. Menos errores.
- **Eficiente:** uso adecuado de recursos (memoria, tiempo de ejecución).

Un lenguaje que cumple con estas características es Java:

- Simplicidad y legibilidad: sintaxis clara y estructurada.
- Claridad en los bindings: tipado estático y explícito.
- Confiabilidad: manejo de excepciones y control automático de memoria.
- Soporte: amplio sistema de herramientas, librerías y comunidad.
- Abstracción: permite programación orientada a objetos.
- Ortogonalidad: sus características se combinan de forma consistente.
- Eficiente: buen rendimiento gracias a la máquina virtual (JVM).

4. Java:

- Tipos de expresiones:
 1. Aritméticas: cálculos matemáticos. Ejemplo: $a+b$
 2. Relacionales: comparan valores. Ej.: $a > b$
 3. Lógicas: combinan condiciones. Ej.: $\&\&$, $\|\|$
 4. De asignación: asignan valores a variables. Ej.: $x = 5$
 5. De incremento/decremento: $x++$, $--y$
 6. Condicional: condición ? valor1 : valor2
 7. De objetos: creación y uso de objetos, `new Clase()`, acceso a métodos y atributos.
- Facilidades provistas para la organización del programa:
Java ofrece varias herramientas para estructurar programas:
 1. Clases y objetos: base de la programación orientada a objetos.
 2. Paquetes: agrupan clases relacionadas.

3. Métodos: permiten dividir el programa en funciones reutilizables.
4. Interfaces: definen comportamientos sin implementación
5. Herencia y polimorfismo: reutilización y extensión código.
6. Modularidad: separación en diferentes archivos y componentes.

- Atributos que posee y no posee Java:

Posee:

1. Simplicidad y legibilidad: sintaxis clara y estructurada
2. Claridad en los bindings: tipado estático
3. Confiabilidad: manejo de excepciones y recolección automática de basura
4. Soporte: gran comunidad, documentación y herramientas.
5. Abstracción: fuerte soporte a OOP
6. Ortogonalidad: muchas características consistentes entre sí

No posee:

1. Simplicidad total: es más verboso que lenguajes modernos como Python
2. Ortogonalidad completa: algunas reglas son inconsistentes, como la diferencia entre tipos primitivos y objetos
3. Eficiencia máxima: no es tan eficiente como C porque usa máquina virtual
4. Flexibilidad en bajo nivel: no permite manipulación directa de memoria.

5. ADA

- Tipos de datos: tipado fuerte y estricto, lo que mejora la confiabilidad.
 - Tipos básicos: Integer, Float, Boolean, Character
 - Tipos definidos por el usuario
 - Subtipos (restricciones sobre tipos existentes)
 - Tipos enumerados
 - Tipos derivados (creación de nuevos tipos a partir de otros)
- Tipos abstractos de datos - Paquetes: utiliza paquetes para encapsular datos y operaciones.
 - Separan interfaz de implementación
 - Permiten ocultamiento de información (encapsulamiento)
 - Facilitan la modularidad y reutilización
- Estructuras de datos: permite definir estructuras complejas de forma segura
 - Arrays con límites definidos

- Records
- Tipos compuestos definidos por el usuario
- Acceso a estructuras dinámicas mediante punteros
- Manejo de excepciones: manejo de errores
 - Uso de bloques exception
 - Excepciones predefinidas
 - Posibilidad de definir excepciones propias
- Manejo de concurrencia: el punto más fuerte de Ada
 - Tareas: unidades de ejecución concurrente
 - Rendezvous: comunicación y sincronización entre tareas
 - Protected objects: control seguro del acceso a recursos compartidos

JAVA

6. Java fue creado para desarrollar software portable para dispositivos electrónicos. Luego se orientó a aplicaciones web y empresariales. Introdujo varios cambios a la web:

- Contenido dinámico: permite que las páginas web no fueran solo estáticas
- Applets: pequeños programas que se ejecutaban en el navegador
- Interactividad: mejoro la experiencia del usuario en la web
- Base para aplicaciones empresariales web (servidores, backend)

Java es independiente de la plataforma porque:

- El código Java se compila a bytecode
- Ese bytecode se ejecuta en la JVM
- Cada sistema operativo tiene su propia JVM

7. Java se basa en:

- C : sintaxis básica, uso de llaves, operadores, estructuras como if, while, for. Estilo imperativo
- C++ : POO (clases, herencias, encapsulamiento). Estructura general del lenguaje. Pero Java simplifica C++ eliminando cosas complejas como punteros explícitos y herencia múltiple.
- Smalltalk: “todo es un objeto” (aunque en Java no es 100% así). Enfoque fuerte en orientación a objetos.

8. Applets → pequeños programas Java diseñados para ejecutarse dentro del navegador web. Se descargan desde un servidor y se ejecutan en la JVM del cliente.

Se usan para interactividad en páginas web, como animaciones, juegos o formularios dinámicos.

Servlets → programas Java que corren en el servidor. Generan contenido dinámico para la web (HTML, JSON, etc). Se usan para procesar peticiones HTTP, interactuar con bases de datos y manejar la lógica de aplicaciones web.

C

9. Estructura de un programa en C:

1. Directivas de preprocesador: #include, #define
2. Declaración de funciones y variables globales
3. Función main(): punto de partida del programa
4. Otras funciones: definidas fuera de main()

```
#include <archivo>
```

```
int suma(int a, int b) { return a + b; }
```

```
int main() {  
    printf(" resultado", suma);  
    return 0;  
}
```

No se pueden definir funciones dentro de otras funciones. Todas las funciones deben declararse fuera del main() o como prototipos antes de usarlas.

10. Manejo de expresiones (cualquier combinación de constantes, variables, operadores y llamadas a funciones que produce un valor).

1. Tipos:

- Aritméticas: `int suma = a + b;`
`float promedio = total / cantidad;`
- Relacionales: comparaciones que devuelven 1 (true) o 0 (false)
`int esMayor = edad > 18;`
- Lógicas: combinaciones de condiciones con &&, ||, !
`int valido = (edad > 18) && (dni != 0);`
- Asignación: asigna valores a variables
`x = 5;`
- Incremento / decremento: `j++ ; i --;`
- Condicional (ternaria): `int max = (a > b) ? a : b;`

- De función: el valor retornado por una función puede ser parte de una expresión. `int resultado = suma(3, 4);`
2. Prioridad y orden de evaluación: sigue reglas estrictas de precedencia y asociatividad de operadores. Usa paréntesis para controlar el orden de evaluación.
3. Anidamiento de expresiones: por ejemplo, `int resultado = (a + b) * (c - d) / 2;`

Python - RUBY - PHP

11. Python

- **Tipos de programas:**
 - Aplicaciones de escritorio
 - Scripts y automatización
 - Ciencia de datos y machine learning
 - Desarrollo web (Django, Flask)
 - Juegos y prototipos
- **Paradigma:**
 - Multiparadigma: **imperativo, orientado a objetos, funcional**
- **Características del paradigma:**
 - Clases y objetos → OOP
 - Funciones como ciudadanos de primera clase → Funcional
 - Bloques secuenciales y control de flujo → Imperativo

Ruby

- **Tipos de programas:**
 - Desarrollo web (Rails)
 - Automatización y scripting
 - Prototipos y aplicaciones de software general
- **Paradigma:**
 - Multiparadigma: **orientado a objetos, imperativo, funcional limitado**
- **Características del paradigma:**
 - Todo en Ruby es un objeto → OOP pura
 - Sintaxis muy legible y expresiva
 - Posibilidad de definir bloques y closures → Funcional

PHP

- **Tipos de programas:**
 - Desarrollo web y backend
 - Generación dinámica de páginas web
 - Sistemas de gestión de contenido (WordPress, Drupal)
- **Paradigma:**

- Multiparadigma: **imperativo, orientado a objetos, scripting**
- **Características del paradigma:**
 - Clases y objetos (OOP)
 - Variables y funciones sueltas → Imperativo
 - Scripts ejecutables directamente en servidor web

12. Python

- **Tipado:** Dinámico y fuerte (no necesita declarar tipos, pero no permite operaciones entre tipos incompatibles)
- **Organización:** Módulos y paquetes, clases
- **Sintaxis:** Muy legible, indentación significativa
- **Gestión de memoria:** Automática
- **Otros:** Multiparadigma (imperativo, OOP, funcional)

Ruby

- **Tipado:** Dinámico y fuerte
- **Organización:** Todo es un objeto; uso de clases, módulos y mixins
- **Sintaxis:** Muy expresiva y legible
- **Gestión de memoria:** Automática
- **Otros:** Soporta bloques y closures; multiparadigma (OOP, imperativo, funcional limitado)

PHP

- **Tipado:** Dinámico, no tan estricto (lo que permite conversiones automáticas)
- **Organización:** Scripts sueltos, funciones, clases y namespaces
- **Sintaxis:** Similar a C y Perl
- **Gestión de memoria:** Automática
- **Otros:** Orientado a web; ejecución en servidor; multiparadigma (imperativo, OOP, scripting)

Gobstones

- **Tipado:** Fuerte y estático (se declara al crear variables)
- **Organización:** Programas secuenciales o con procedimientos
- **Sintaxis:** Visual y sencilla (pensado para educación)
- **Gestión de memoria:** Automática
- **Otros:** Lenguaje educativo; diseñado para aprender algoritmos y estructuras de control

Processing

- **Tipado:** Dinámico y flexible (aunque soporta tipos explícitos como int, float, boolean)
- **Organización:** Funciones y clases, orientado a sketches (programas gráficos)
- **Sintaxis:** Similar a Java, simplificada para gráficos
- **Gestión de memoria:** Automática
- **Otros:** Orientado a gráficos, visualización y multimedia; multiparadigma (imperativo y OOP)

Javascript

13. Es multiparadigma:

- Imperativo: secuencia de instrucciones paso a paso
- Orientado a objetos: objetos y prototipos
- Basado en eventos: usado en interacción con el servidor
- Funcional

Tipo de lenguaje:

- Lenguaje de alto nivel: cercano al humano y abstrae detalles del hardware
- Lenguaje interpretado: ejecutado por el motor del navegador (antes que compilado)
- Lenguaje de scripting: diseñado para automatizar tareas, interactuar con páginas web y manipular el DOM

14. Otras características:

- Tipado de datos: es dinámico, no necesita declarar el tipo de variable.
- Manejo de excepciones: permite capturar errores en tiempo de ejecución.
- Variables y alcance: var (global o función), let (bloque) y const (bloque, no permite reasignación).
- Las funciones pueden asignarse a variables, pasarse como parámetros y retorna desde otras funciones.
- Utiliza prototipos para herencia (POO).
- Asincronía y eventos: callbacks, promesas y async/await para manejar tareas asíncronas
- Convierte automáticamente valores entre tipos según contexto. Ejemplo:
"5" - 2 → 3, "5" + 2 → "52"
- Manipulación del DOM
- JSON